

Practical no 6

Theory : The Dining Philosophers Problem is a classical synchronization problem in Operating Systems. It consists of N philosophers sitting around a circular table with one chopstick placed between each pair. A philosopher needs both the left and right chopsticks to eat, and after eating, he puts them down and starts thinking again.

If all philosophers try to pick up one chopstick at the same time, a deadlock situation may occur. To prevent this, synchronization techniques like semaphores are used to ensure mutual exclusion and proper sharing of resources without conflict.

Output:

```
ANUJ VERMA@DESKTOP-VE8LIPH MSYS ~
$ cd

ANUJ VERMA@DESKTOP-VE8LIPH MSYS ~
$ nano philo.c

ANUJ VERMA@DESKTOP-VE8LIPH MSYS ~
$ gcc philo.c -o philo

ANUJ VERMA@DESKTOP-VE8LIPH MSYS ~
$ ./philo
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 2 is eating
Philosopher 2 finished eating
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 3 finished eating
Philosopher 1 finished eating
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 4 is eating
Philosopher 4 finished eating

ANUJ VERMA@DESKTOP-VE8LIPH MSYS ~
$ nano philo.c

ANUJ VERMA@DESKTOP-VE8LIPH MSYS ~
$
```

```
GNU nano 2.7 philo.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[10];

void* philosopher(void* num) {
    int id = *(int*)num;

    printf("Philosopher %d is thinking\n", id);
    sleep(1);

    // Deadlock prevention
    if (id == N - 1) {
        sem_wait(&chopstick[(id + 1) % N]);
        sem_wait(&chopstick[id]);
    } else {
        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id + 1) % N]);
    }

    printf("Philosopher %d is eating\n", id);
    sleep(2);

    sem_post(&chopstick[id]);
    sem_post(&chopstick[(id + 1) % N]);

    printf("Philosopher %d finished eating\n", id);
    return NULL;
}

int main() {
    pthread_t phil[N];
    int ids[N];

    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(phil[i], NULL);
    }

    return 0;
}
```